



Outline

Definitions

Uniform Quantization

Examples

Rate-distortion curve

Optimal Scalar Quantization

Predictive scalar quantization







Prediction gain

Prediction Error = Signal Error :

$$q(n) = y(n) - \hat{y}(n) = x(n) - v(n) - \hat{x}(n) + v(n) = \bar{q}(n)$$

Thus the goal of predictive quantization (PQ) is to minimize the distortion of y
The coding gain is:

$$\text{SNR}_p = 10 \log_{10} \frac{\sigma_X^2}{D} = 10 \log_{10} \frac{\sigma_X^2}{\sigma_Y^2} + 10 \log_{10} \frac{\sigma_Y^2}{D} = G_P + G_Q$$

Prediction is effective if and only if the prediction error has a smaller variance than the original signal



Go to wooclap.com with the code **PBTFUO**

$$X(n) \sim \mathcal{N}(0, \sigma^2)$$

$$V(n) = X(n - 1)$$

$$\mathbb{E}[X(n)X(m)] = \sigma^2 \rho^{|n-m|}$$

$\rho : G_P > 0 ?$





Predictors

- ▶ Linear Predictors are simple and moreover optimal for Gaussian RV

$$v(n) = - \sum_{i=1}^P a_i x_{n-i} \quad \text{Filter with } P \text{ parameters}$$

$$y(n) = x(n) - v(n) = \sum_{i=0}^P a_i x_{n-i} \quad \text{Prediction error}$$

- ▶ with $a_0 = 1$.
- ▶ y is the result of filtering x with

$$A(z) = 1 + a_1 z^{-1} + \dots + a_P z^{-P}$$

- ▶ Optimal filter: minimization of σ_Y^2





Predictor selection

$$\begin{aligned}\sigma_Y^2 &= E\{X^2(n)\} + 2 \sum_{i=1}^P a_i E\{X(n)X(n-i)\} + \sum_{i=1}^P \sum_{j=1}^P a_i a_j E\{X(n-i)X(n-j)\} \\ &= \sigma_X^2 + 2\mathbf{r}^t \mathbf{a} + \mathbf{a}^t \mathbf{R}_X \mathbf{a}\end{aligned}$$

with:

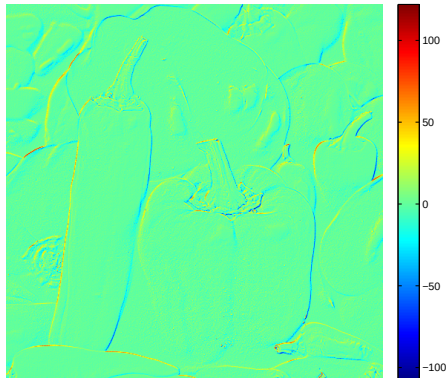
$$\mathbf{r} = [r_X(1) \dots r_X(P)] \quad \mathbf{R}_X = \begin{bmatrix} r_X(0) & r_X(1) & r_X(2) & \dots & r_X(P-1) \\ r_X(1) & r_X(0) & r_X(1) & \dots & r_X(P-2) \\ r_X(2) & r_X(1) & r_X(0) & \dots & r_X(P-3) \\ \dots & \dots & \dots & \dots & \dots \\ r_X(P-2) & r_X(P-3) & r_X(P-4) & \dots & r_X(1) \\ r_X(P-1) & r_X(P-2) & r_X(P-3) & \dots & r_X(0) \end{bmatrix}$$

$$r_X(k) = E\{X(n)X(n-k)\}$$





Predictive quantization: example



For the “peppers” image (grayscale), we compute the prediction error using a second order linear predictor:

$$\hat{f}_{n,m} = af_{n-1,m} + bf_{n,m-1}$$

The predictor of $f_{n,m}$ is a weighted average of the top and the left neighbors.

a	b	σ_Y^2
0	0	2902.7
1/2	1/2	78.7
0.449	0.546	78.4

The first row correspond to no prediction: thus $\sigma_X^2 = 2902.7$

The second row show the distortion of the simplest predictor: the average of the two neighbors. The distortion is reduced by a factor greater than 37

Using the optimal predictor, computed with the formulas shown in the previous slide, only slightly improves the performance



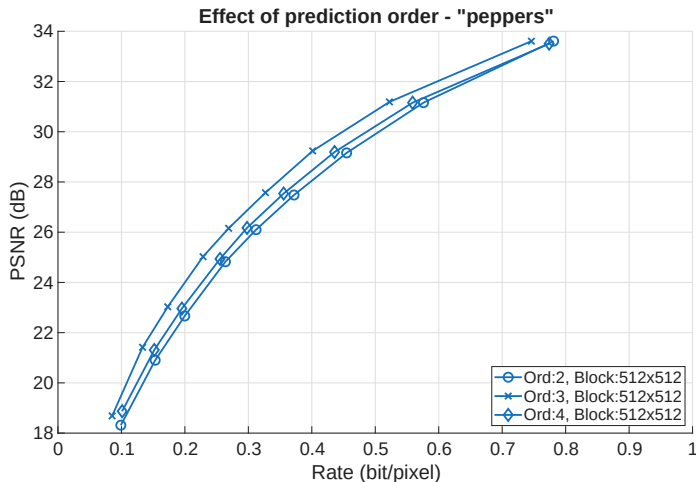
Impact of Filter Order on Prediction Gain

In global DPCM, the predictor $v(n)$ uses a linear combination of N causal neighbors.

- ▶ **Order Selection:** Increasing the order from 2 to 4 typically improves the **Prediction Gain** ($G_p = \sigma_X^2 / \sigma_Y^2$), leading to a more "whitened" residual.
- ▶ **Screening Effect:** The benefit of including "distant" pixels diminishes rapidly. In typical image statistics, the immediate neighbors capture most of the mutual information. Distant pixels are often redundant as their correlation with the target is already "screened" by the closer neighbors.
- ▶ **Complexity vs. Performance:** Higher orders increase computational overhead with negligible PSNR gains for most natural textures.



Rate-Distortion Analysis: Filter Order

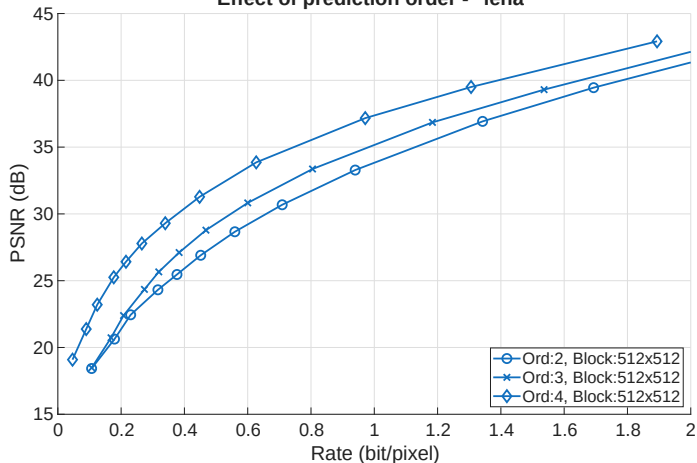


Homogeneous image with some background noise



Rate-Distortion Analysis: Filter Order

Effect of prediction order - "lena"



Clean image with different areas



Local Adaptation: The Block-wise Approach

Images are **non-stationary** signals; a single global filter is suboptimal for varying textures (e.g., edges vs. smooth areas). **The Strategy:** Divide the image into $M \times M$ blocks and

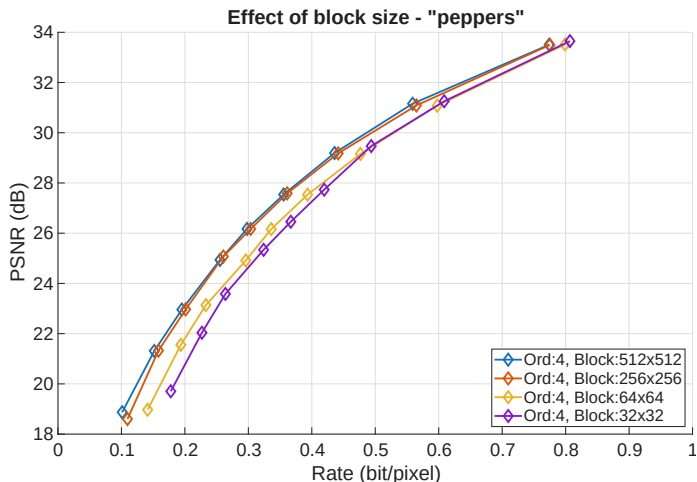
compute the **locally optimal** filter for each.

The Side Information Trade-off Total Rate (R_{total}) is now increased by $\frac{N \cdot B}{M^2}$ Where N is the filter order and B is the bits per coefficient (e.g., 16).

- ▶ **Small Blocks:** Excellent tracking of local statistics, but massive **overhead** from filter coefficients.
- ▶ **Large Blocks:** Minimal overhead, but the predictor fails to adapt to local features.



Rate-Distortion Analysis: Block size

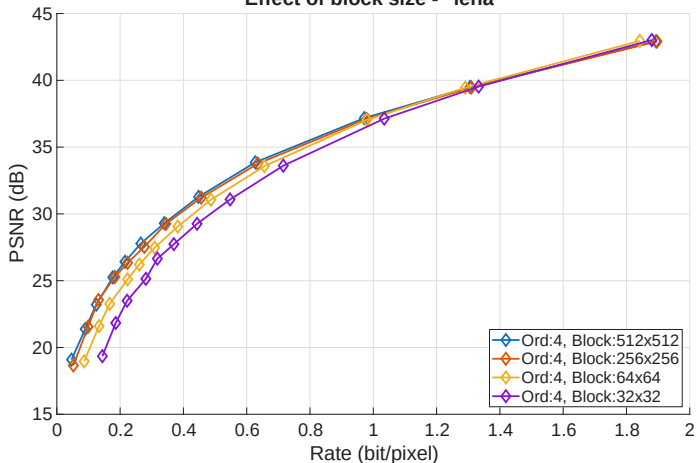


Homogeneous image with some background noise



Rate-Distortion Analysis: Block size

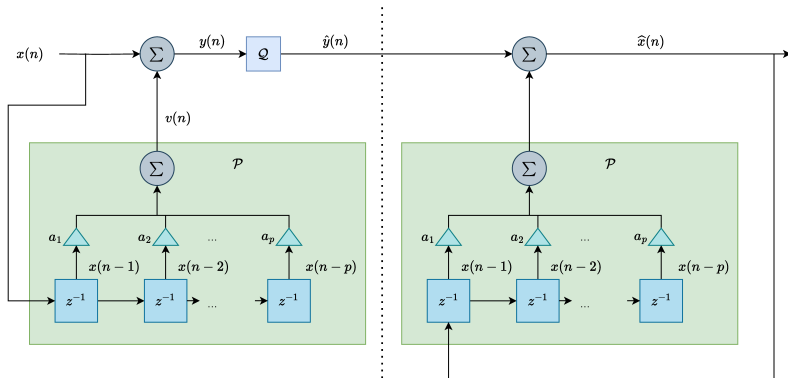
Effect of block size - "lena"



Clean image with different areas

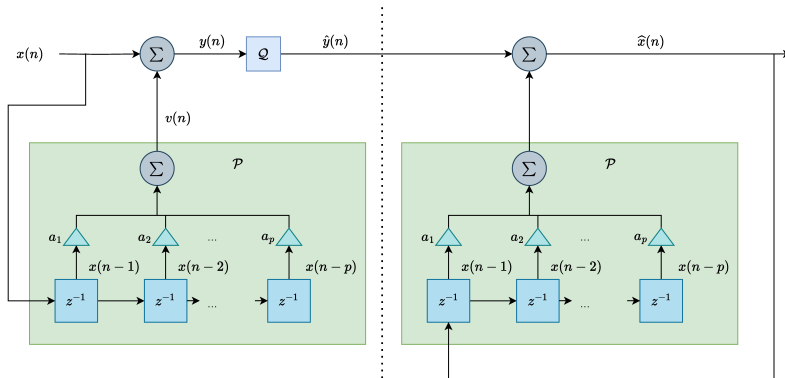


Coding/decoding scheme





Coding/decoding scheme



WRONG! The predictor operator is the same at the encoder and decoder but the input is different: $x \neq \hat{x}$! This causes **drift**.



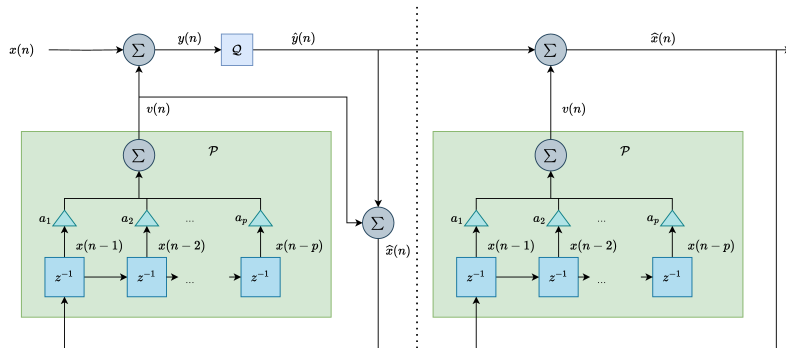
Predictive Quantization Drift Example

- ▶ Simple predictive scheme: prediction is the previous value
- ▶ 3-bit quantizer (7 levels): $[-9, -6, -3, 0, 3, 6, 9]$
- ▶ Encoder uses non-quantized values for prediction
- ▶ Decoder uses quantized (reconstructed) values for prediction

Step	Original Signal	Encoder			Decoder		
		Prediction	Error	Quantized Error	Prediction	Received Error	Reconstructed Value
1	10	-	-	9	-	9	9
2	11	10	1	0	9	0	9
3	12	11	1	0	9	0	9
4	13	12	1	0	9	0	9
5	14	13	1	0	9	0	9
6	18	14	4	3	9	3	12
7	21	18	3	3	12	3	15
8	18	21	-3	-3	15	-3	12



Coding/decoding scheme



This is the correct scheme: the predictor is fed with the same data (quantized past samples) both at the encoder and the decoder.



Predictive Quantization Drift Example

- ▶ Simple predictive scheme: prediction is the previous value
- ▶ 3-bit quantizer (7 levels): $[-9, -6, -3, 0, 3, 6, 9]$
- ▶ Encoder uses quantized values for prediction
- ▶ Decoder uses quantized (reconstructed) values for prediction

Step	Original Signal	Encoder			Decoder		
		Prediction	Error	Quantized Error	Prediction	Received Error	Reconstructed Value
1	10	-	-	9	-	9	9
2	11	9	2	3	9	3	12
3	12	12	0	0	12	0	12
4	13	12	1	0	12	0	12
5	14	12	2	3	12	3	15
6	18	15	3	3	15	3	18
7	21	18	3	3	18	3	21
8	18	21	-3	-3	21	-3	18

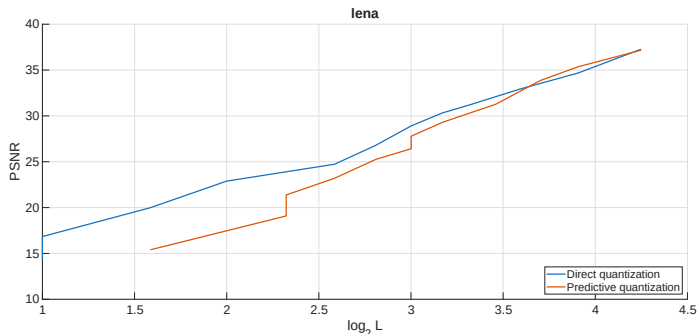


Coding scheme

- ▶ The encoder computes the prediction error $Y = X - V$
- ▶ The predictor V must be computed from data available at the decoder as well
- ▶ Thus the encoder must compute the reconstructed value $\hat{X}$
- ▶ This value is sent into a buffer and used to compute the prediction of future values



Direct vs. Predictive quantization

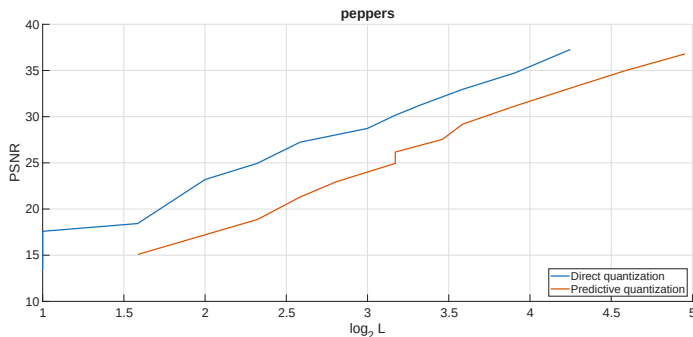


We show here the PSNR of the quantized image vs. the $\log_2 L$, which is our first rate estimation. The result seems very deceiving: apart from a short range of high rates, predictive quantization does not bring the expected gain: how is this possible?

The main explanation is that a practical PQ scheme must use the quantized data to produce a predictor, which is not what we considered in our theoretical analysis.



Direct vs. Predictive quantization



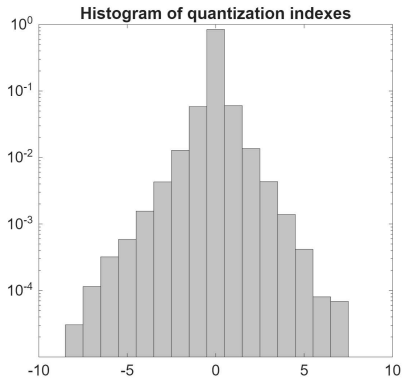
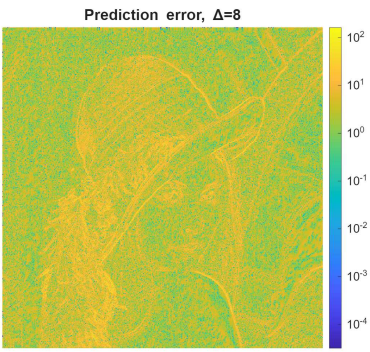
For the **peppers** image things are even worse.

- ▶ At low bitrates, the predictor is computed on heavily quantized data
- ▶ At high bitrates, the quantizer is obliged to reconstruct the additive noise affecting the image

However, the main point is that **the estimation of the rate with the logarithm is too much pessimistic**



PQ prediction error





PQ prediction error

Let us look more closely to the quantized prediction error. In this example, $L = 19$, so the estimated rate is $\log_2 19 \approx 4.25$ bits per sample.

However, we observe that most of the quantization indexes ($\approx 84\%$) are zero. We could use a **variable lenght code** that uses a short codeword for the index 0 and possibly longer codewords for indexes outside $(-3, 3)$, which account for less than 1% of the total.

Encoding the quantizations indexes with non-trivial encoding strategy (called **entropy coding**, as we will see later on) changes dramatically the results





Quantization: wrap-up

- ▶ UQ: $D = K_X \sigma_X^2 2^{-2R}$ for uniform RV or in high res
- ▶ Optimal SQ: similar behavior
- ▶ Both UQ and optimal SQ alone are catastrophic if applied directly on images or sound: the quality drops and the rate is not reduced much
- ▶ **Predictive quantization** can improve dramatically performance but:
 1. Never forget to compute the prediction on **quantized data**, otherwise the decoder will drift
 2. The input signal **must be correlated enough**
 3. **We must use the best encoding strategy to achieve good performance**

The best encoding strategy is **entropy coding**, the subject of the next lesson